

CRUD 완성 — PATCH·오류 처리·집계 API

1. PUT vs PATCH

메서드	의미	동작 방식
PUT	전체 교체	보내지 않은 필드는 null/기본값으로 초기화
PATCH	부분 수정	보낸 필드만 수정, 나머지는 그대로 유지

PUT 방식 — 문제점

`display_name` 만 바꾸고 싶어도 `username` 을 함께 보내야 합니다.

만약 `username` 을 빠뜨리면 null로 초기화될 위험이 있습니다.

```
# PUT /users/{id} - 전체를 보내야 함
{
  "username": "kim",
  "display_name": "김강사" # display_name만 바꾸고 싶어도 username도 보내야 함
}
```

PATCH 방식 — 올바른 설계

```
# PATCH /users/{id} - 바꿀 것만 보내면 됨
{
  "display_name": "김강사" # username은 보내지 않아도 됨
}
```

2. PATCH 구현 — Optional 필드와 exclude_unset

PATCH를 위한 Pydantic 스키마는 모든 필드를 `Optional` 로 만듭니다.

```
from typing import Optional

class UserUpdate(BaseModel):
    display_name: Optional[str] = Field(default=None, min_length=1, max_length=50)
    # 보내지 않으면 None, 보내면 값이 들어옴
```

[비추천] model_dump(): 보내지 않은 필드가 None으로 포함됨

```
# 요청: {"display_name": "김강사"}  
data = UserUpdate(display_name="김강사")  
  
# model_dump() - 보내지 않은 필드가 None으로 포함됨  
data.model_dump()  
# {"display_name": "김강사"} ← 이 경우엔 괜찮아 보이지만...
```

[추천] model_dump(exclude_unset=True): 실제로 보낸 필드만

```
# 예: username도 Optional이라면  
# data.model_dump() → {"username": None, "display_name": "김강사"}  
# → Supabase에 username=None으로 업데이트해서 데이터 손상!  
  
# model_dump(exclude_unset=True) - 실제로 보낸 필드만  
data.model_dump(exclude_unset=True)  
# {"display_name": "김강사"} ← username 제외
```

Repository에서 PATCH 구현

```
def update_user(self, user_id: str, data: UserUpdate) -> dict:
    payload = data.model_dump(exclude_unset=True) # 보낸 필드만
    if not payload:
        raise HTTPException(status_code=400, detail="수정할 값이 없습니다")

    result = (
        self.db.table("app_users")
        .update(payload)
        .eq("id", user_id)
        .execute()
    )
    return self.one_or_404(result.data, "사용자를 찾을 수 없습니다")
```

update()는 수정된 행을 기본 반환

supabase-py 2.x 의 `update()` 는 수정된 행을 `result.data` 로 기본 반환합니다.

```
result = self.db.table("app_users").update(payload).eq("id", user_id).execute()  
# result.data → [{"id": "...", "display_name": "김강사", ...}]
```

쓰기 쿼리 뒤에 `.select("*")` 를 연결하면 `AttributeError` 가 발생하므로 바로 `.execute()` 를 호출합니다.

3. Query 파라미터로 메시지 개수 제한

```
from fastapi import Query

@app.get("/conversations/{conversation_id}/messages")
def list_messages(
    conversation_id: str,
    limit: int = Query(default=50, ge=1, le=100),
    #           ↑           ↑           ↑           ↑
    #           기본 50    최소 1    최대 100
    repo: SupabaseRepository = Depends(get_repo),
):
    return repo.list_messages(conversation_id, limit)
```

Swagger에서 `limit` 쿼리 파라미터가 자동으로 UI에 표시됩니다.

```
GET /conversations/{id}/messages?limit=20
```

4. 집계 API — Summary

사용자가 몇 개의 대화방과 메시지를 가지고 있는지 요약합니다.

```
def summary(self, user_id: str) -> ChatSummary:
    conversations = self.list_conversations(user_id)
    message_count = 0
    for conv in conversations:
        ...

    return ChatSummary(
        user_id=user_id,
        conversation_count=len(conversations),
        message_count=message_count,
    )
```

5. 서버 실행 및 Swagger 실습

가상환경 생성

```
# pyproject.toml과 uv.lock 기준으로 의존성 설치  
uv sync
```

서버 실행

```
# 1. .env 파일에 실제 키 입력  
# 2. 서버 실행  
uvicorn main:app --reload --port 8001
```

Swagger UI 실습 (<http://localhost:8001/docs>)

각 API에서 Try it out을 누른 뒤 아래 순서대로 입력합니다.

이번 실습에서는 앞 요청의 응답으로 받은 ID를 다음 요청에서 계속 사용합니다.

이름	저장할 값
USER_ID	사용자 생성 응답의 id
CONVERSATION_ID	대화방 생성 응답의 id

실습 1. 사용자 생성

POST /users → Try it out → Body 입력 → Execute

```
{  
  "username": "swagger05",  
  "display_name": "스웨거 학생"  
}
```

- 201 Created 확인
- 응답의 id 를 USER_ID 로 복사
- 같은 username이 이미 있다면 숫자를 바꾸어 다시 실행

POST

/users Create User

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "username": "swagger05",
  "display_name": "스웨거 학생"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "swagger05",
    "display_name": "스웨거 학생"
  }'
```

Request URL

`http://localhost:8001/users`

Server response

Code	Details
------	---------

201

Response body

```
{
  "id": "5ef43385-0ca2-433a-a397-bd3b2c261b1c",
  "username": "swagger05",
  "display_name": "스웨거 학생",
  "created_at": "2026-06-21T06:32:08.798074+00:00"
}
```



Download

Response headers

```
content-length: 150
content-type: application/json
date: Sun, 21 Jun 2026 06:32:07 GMT
server: uvicorn
```

실습 2. PATCH로 사용자 일부 수정

PATCH /users/{user_id} → user_id 에 USER_ID 입력 → Body 입력 → Execute

```
{  
  "display_name": "FastAPI 수강생"  
}
```

→ 200 OK 와 변경된 display_name 확인

→ 요청에 username 을 보내지 않았지만 기존 값 "swagger05" 가 유지되는지 확인

app.patch() 가 PATCH 요청을 연결하고, exclude_unset=True 가 보내지 않은 필드를 UPDATE에서 제외합니다.

PATCH

/users/{user_id} Update User

Parameters

Cancel

Reset

Name	Description
user_id ★ required string <i>(path)</i>	5ef43385-0ca2-433a-a397-bd3b2c261b1c

Request body required

application/json

Edit Value | Schema

```
{  "display_name": "FastAPI 수강생"}  
```

Execute

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "display_name": "FastAPI 수강생"
  }'
```

Request URL

http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c

Server response

Code

Details

200

Response body

```
{
  "id": "5ef43385-0ca2-433a-a397-bd3b2c261b1c",
  "username": "swagger05",
  "display_name": "FastAPI 수강생",
  "created_at": "2026-06-21T06:32:08.798074+00:00"
}
```



Download

Response headers

```
content-length: 151
content-type: application/json
date: Sun, 21 Jun 2026 06:44:29 GMT
server: uvicorn
```

실습 3. 수정할 값이 없을 때 400 확인

PATCH /users/{user_id} → user_id 에 USER_ID 입력 → Body에 {} 입력 → Execute
→ 400 Bad Request 확인

```
{  
  "detail": "수정할 값이 없습니다"  
}
```

{ } 는 Pydantic 스키마를 통과하지만 `exclude_unset=True` 의 결과가 빈 딕셔너리이므로 Repository가 400을 반환합니다.

PATCH

/users/{user_id} Update User

⌵

Parameters

Cancel

Reset

Name	Description
user_id ★ required string (path)	5ef43385-0ca2-433a-a397-bd3b2c261b1c

Request body

required

application/json

⌵

Edit Value

Schema

{}

Execute

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{}'
```

Request URL

```
http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c
```

Server response

Code	Details
------	---------

400	Error: Bad Request
-----	--------------------

Undocumented

Response body

```
{
  "detail": "수정할 값이 없습니다"
}
```

Response headers

```
content-length: 42
content-type: application/json
date: Sun, 21 Jun 2026 06:45:55 GMT
server: uvicorn
```

실습 4. 존재하지 않는 사용자 수정 시 404 확인

PATCH /users/{user_id} → 아래 값을 user_id 에 입력 → Body 입력 → Execute

```
00000000-0000-0000-0000-000000000000
```

```
{  
  "display_name": "없는 사용자"  
}
```

→ 404 Not Found 와 {"detail": "사용자를 찾을 수 없습니다"} 확인

→ UPDATE 결과가 빈 목록이면 one_or_404() 가 404로 변환

PATCH

/users/{user_id} Update User

Parameters

Cancel

Reset

Name	Description
user_id ★ required string (path)	00000000-0000-0000-0000-000000000000

Request body

required

application/json

Edit Value

Schema

```
{  "display_name": "없는 사용자"}}
```

Execute

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:8001/users/00000000-0000-0000-0000-000000000000' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "display_name": "없는 사용자"
  }'
```



Request URL

http://localhost:8001/users/00000000-0000-0000-0000-000000000000

Server response

Code

Details

404

Error: Not Found

Undocumented

Response body

```
{
  "detail": "사용자를 찾을 수 없습니다"
}
```



Download

Response headers

```
content-length: 49
content-type: application/json
date: Sun, 21 Jun 2026 06:47:31 GMT
server: uvicorn
```

실습 5. 대화방 생성

POST /users/{user_id}/conversations → user_id 에 USER_ID 입력 → Body 입력 →

Execute

```
{  
  "title": "PATCH와 집계 실습"  
}
```

→ 201 Created 확인

→ 응답의 id 를 CONVERSATION_ID 로 복사

POST

/users/{user_id}/conversations

Create Conversation

Parameters

Cancel

Reset

Name	Description
user_id ★ required string <i>(path)</i>	5ef43385-0ca2-433a-a397-bd3b2c261b1c

Request body required

application/json ▼

Edit Value | Schema

```
{  "title": "PATCH와 집계 설습"}  
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c/conversations' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "PATCH와 집계 실습"
  }'
```

Request URL

```
http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c/conversations
```

Server response

Code	Details
------	---------

201	
-----	--

Response body

```
{
  "id": "a99fef9b-bb27-45ad-ac75-744221875e0f",
  "user_id": "5ef43385-0ca2-433a-a397-bd3b2c261b1c",
  "title": "PATCH와 집계 실습",
  "created_at": "2026-06-21T06:49:29.417259+00:00"
}
```

[Download](#)

Response headers

```
content-length: 175
content-type: application/json
date: Sun, 21 Jun 2026 06:49:28 GMT
server: uvicorn
```

실습 6. 대화방 제목 수정

PATCH /conversations/{conversation_id} → conversation_id 에 CONVERSATION_ID 입력
→ Body 입력 → Execute

```
{  
  "title": "Swagger 실습 완료하기"  
}
```

→ 200 OK 와 변경된 title 확인

현재 ConversationUpdate 의 수정 가능 필드는 title 하나이며 필수이므로
exclude_unset=True 를 사용하지 않습니다.

PATCH

/conversations/{conversation_id}

Update Conversation



Parameters

Cancel

Reset

Name	Description
conversation_id ★ required string (path)	a99fef9b-bb27-45ad-ac75-744221875e0f

Request body

required

application/json

▼

Edit Value

Schema

```
{  "title": "Swagger 실습 완료하기"}  
```

Execute

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:8001/conversations/a99fef9b-bb27-45ad-ac75-744221875e0f' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Swagger 실습 완료하기"
  }'
```



Request URL

```
http://localhost:8001/conversations/a99fef9b-bb27-45ad-ac75-744221875e0f
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "id": "a99fef9b-bb27-45ad-ac75-744221875e0f",
  "user_id": "5ef43385-0ca2-433a-a397-bd3b2c261b1c",
  "title": "Swagger 실습 완료하기",
  "created_at": "2026-06-21T06:49:29.417259+00:00"
}
```

[Download](#)

Response headers

```
content-length: 180
content-type: application/json
date: Sun, 21 Jun 2026 06:51:01 GMT
server: uvicorn
```

실습 7. 메시지 3개 생성

POST /conversations/{conversation_id}/messages 에서 conversation_id 에 CONVERSATION_ID 를 입력하고, Body만 바꾸어 세 번 실행합니다.

```
{"role": "user", "content": "첫 번째 메시지"}
```

POST /conversations/{conversation_id}/messages Create Message

Parameters

Cancel

Reset

Name	Description
conversation_id ★ required string <i>(path)</i>	a99fef9b-bb27-45ad-ac75-744221875e0f

Request body required

application/json

Edit Value | Schema

```
{  
  "role": "user",  
  "content": "첫 번째 메시지"  
}
```

Execute

```
{"role": "assistant", "content": "두 번째 메시지"}
```

POST /conversations/{conversation_id}/messages Create Message ^

Parameters

Cancel

Reset

Name	Description
conversation_id ★ required string <i>(path)</i>	a99fef9b-bb27-45ad-ac75-744221875e0f

Request body required

application/json

Edit Value

Schema

```
{  
  "role": "assistant",  
  "content": "두 번째 메시지"  
}
```

Execute

```
{"role": "user", "content": "세 번째 메시지"}
```

POST

/conversations/{conversation_id}/messages

Create Message

Parameters

Cancel

Reset

Name	Description
conversation_id ★ required	
string (path)	a99fef9b-bb27-45ad-ac75-744221875e0f

Request body required

application/json

Edit Value | Schema

```
{  
  "role": "user",  
  "content": "세 번째 메시지"  
}
```

Execute

실습 8. Query 파라미터로 2개만 조회

```
GET /conversations/{conversation_id}/messages
```

- `conversation_id` : `CONVERSATION_ID`
- `limit` : `2`

→ **Execute** → `200 OK` 확인

→ 메시지가 오래된 순서로 정확히 2개만 반환되는지 확인

GET /conversations/{CONVERSATION_ID}/messages?limit=2

GET

/conversations/{conversation_id}/messages List Messages

^

Parameters

Cancel

Name	Description
conversation_id ★ required string <i>(path)</i>	a99fef9b-bb27-45ad-ac75-744221875e0f
limit integer <i>(query)</i>	2 maximum: 100 minimum: 1

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8001/conversations/a99fef9b-bb27-45ad-ac75-744221875e0f/messages?limit=2' \
  -H 'accept: application/json'
```



Request URL

```
http://localhost:8001/conversations/a99fef9b-bb27-45ad-ac75-744221875e0f/messages?limit=2
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
[
  {
    "id": "9ce1e581-d7e7-4335-8a79-bcda86d935f2",
    "conversation_id": "a99fef9b-bb27-45ad-ac75-744221875e0f",
    "role": "user",
    "content": "첫 번째 메시지",
    "created_at": "2026-06-21T06:52:51.997239+00:00"
  },
  {
    "id": "9f62a819-0620-4502-93c4-96df389fc1a3",
    "conversation_id": "a99fef9b-bb27-45ad-ac75-744221875e0f",
    "role": "assistant",
    "content": "두 번째 메시지",
    "created_at": "2026-06-21T06:53:43.757569+00:00"
  }
]
```

[Download](#)

Response headers

```
content-length: 402
content-type: application/json
date: Sun, 21 Jun 2026 06:56:19 GMT
server: uvicorn
```

실습 9. limit 범위 검증 실패

GET /conversations/{conversation_id}/messages

- conversation_id : CONVERSATION_ID
- limit : 0

→ **Execute** → 422 Unprocessable Entity 확인

→ 오류의 loc 에 "query", "limit" 가 표시되는지 확인

Query(50, ge=1, le=100) 이므로 1보다 작거나 100보다 큰 값은 엔드포인트 실행 전에 거부됩니다.

GET

/conversations/{conversation_id}/messages

List Messages

^

Parameters

Cancel

Name	Description
conversation_id ★ required string <i>(path)</i>	a99fef9b-bb27-45ad-ac75-744221875e0f
limit integer <i>(query)</i>	0 maximum: 100 minimum: 1

Please correct the following validation errors and try again.

- For 'limit': Value must be greater than or equal to 1.

Execute

실습 10. 사용자별 대화·메시지 집계

GET /users/{user_id}/summary → user_id 에 USER_ID 입력 → Execute
→ 200 OK 와 다음 형태의 응답 확인

The screenshot shows a REST client interface with a blue header bar. The header bar contains a 'GET' button, the URL '/users/{user_id}/summary', and the text 'Get Summary'. A small upward arrow is on the right. Below the header is a 'Parameters' section with a 'Cancel' button on the right. The parameters are listed in a table with columns 'Name' and 'Description'. There is one parameter: 'user_id' with a red star and 'required' text, type 'string', and path notation '{path}'. The value '5ef43385-0ca2-433a-a397-bd3b2c261b1c' is entered in a text box. At the bottom is a large blue 'Execute' button.

Name	Description
user_id ★ required string {path}	5ef43385-0ca2-433a-a397-bd3b2c261b1c

이 실습에서 만든 데이터만 있다면 대화방은 1개, 메시지는 3개입니다. 같은 사용자로 추가 데이터를 만들었다면 집계 값은 그만큼 증가합니다.

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c/summary' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8001/users/5ef43385-0ca2-433a-a397-bd3b2c261b1c/summary
```

Server response

Code	Details
200	Response body<pre>{ "user_id": "5ef43385-0ca2-433a-a397-bd3b2c261b1c", "conversation_count": 1, "message_count": 3 }</pre> Response headers<pre>content-length: 91 content-type: application/json date: Sun, 21 Jun 2026 07:00:52 GMT server: uvicorn</pre>